

教程：GitHub 与 GitHub Desktop 新手入门

目录

GitHub 与 GitHub Desktop

新手教程	2
目录与学习目标	2
学完后你应该能够	2
1. 先理解：同一个项目有三个位置	3
用“云盘”作类比	3
2. 六个核心概念	3
Commit 与 Push 为什么是两步？	3
Fetch 与 Pull 的区别	3
3. 第一次准备项目	4
步骤 A：Fork 原作者仓库	4
步骤 B：用 GitHub Desktop Clone	4
步骤 C：确认没有选错	4
4. GitHub Desktop 主界面	4
顶部三个区域	4
Changes 页面	4
History 页面	4
左下角提交区域	4
5. 推荐的日常操作流程	5
开始工作前	5
修改完成后	5
一次提交应该多大？	5
6. 分支：安全的实验空间	5
在 GitHub Desktop 创建分支	5
分支工作流程	6
7. Pull Request：请求合并修改	6
向原作者贡献的完整流程	6
8. 同步原作者后续更新	6
简单方法：GitHub 网站 Sync fork	6
同步前的注意事项	6
9. 冲突：两边改了同一处	7
看到冲突时先做什么	7
10. 撤销与恢复	7
为什么推荐 Revert？	7
11. 文件、空目录与 .gitignore	7
为什么空文件夹不会上传？	7
.gitignore 的作用	8
绝对不要上传	8
12. 完整跟练：添加实验说明	8
第 1 步：同步	8
第 2 步：创建分支	8
第 3 步：创建文件	8

第 4 步：检查并提交	8
第 5 步：确认结果	9
13. 常见问题	9
修改后为什么 GitHub 网站看不到?	9
GitHub 有更新，电脑为什么没有?	9
为什么我看不到空文件夹?	9
Push 被拒绝怎么办?	9
原作者能看到我的修改吗?	9
可以直接在 main 修改吗?	9
Commit 以后还能改吗?	9
14. 操作速查表	9
最终记忆口诀	9
每次 Push 前检查	10

零基础实践手册

GitHub 与 GitHub Desktop 新手教程

从 Fork、Clone 到 Commit、Push、Pull 和 Pull Request：理解“代码在哪里”，安全地完成第一次协作。

适合：第一次接触 Git/GitHub 的学习者

示例环境：Windows + GitHub Desktop + MATLAB 项目

目录与学习目标

1. 先理解三个位置
2. 六个核心概念
3. 安装和首次准备
4. Fork 与 Clone
5. 认识 GitHub Desktop 界面
6. 标准日常流程
7. 分支的使用
8. Pull Request
9. 同步原作者更新
10. 冲突处理
11. 撤销与恢复
12. 安全与 .gitignore
13. 完整练习
14. 常见问题
15. 速查表

学完后你应该能够

- 判断修改是在本地、自己的 GitHub，还是原作者仓库。
- 安全地执行 Fetch、Pull、Commit 和 Push。
- 使用分支隔离实验，不直接破坏主分支。
- 通过 Pull Request 向原作者提交修改建议。
- 遇到冲突时先保护文件，再逐步解决。

最重要的习惯：操作前确认仓库和分支；修改前先 Pull；提交前仔细看 Changes；提交后记得 Push。

1. 先理解：同一个项目有三个位置

很多新手的困惑都来自“不知道现在操作的是哪一份项目”。

位置	是什么	谁能看到
原作者的 GitHub 仓库	项目的上游原始版本，通常称为 upstream	公开仓库任何人可看；你通常不能直接写
你自己的 GitHub 仓库	通过 Fork 得到的在线副本，通常称为 origin	公开仓库任何人可看，但只有授权者能修
你电脑上的本地仓库	通过 Clone 下载的工作目录	默认只有你能看到

原作者仓库 → Fork → 你的 GitHub 仓库 → Clone → 你的电脑

本地修改不会自动上传。你需要先 **Commit**，再 **Push**。原作者也不会自动获得你的修改；要贡献回原仓库，需要创建 **Pull Request**。

用“云盘”作类比

- Clone：把在线项目下载到电脑。
- Commit：在电脑上制作一个有说明的版本存档。
- Push：把本地存档上传到自己的 GitHub。
- Pull：把 GitHub 的新版本下载并合并到电脑。

区别：Git 不是普通同步盘。它会保留每次提交的作者、时间、说明和差异，因此操作更可追踪，也需要你主动同步。

2. 六个核心概念

概念	一句话解释	常见入口
Fork	把别人的 GitHub 仓库复制到自己账号	GitHub 网站右上角 Fork
Clone	把你的在线仓库首次下载到电脑	File → Clone repository
Commit	把一批本地修改记录为一个版本	Changes 页面左下角
Push	把本地提交上传到 GitHub	顶部 Push origin
Fetch / Pull	检查更新 / 下载并合并更新	顶部 Fetch/Pull origin
Pull Request	请求别人审查并合并你的提交	GitHub 网站 Contribute

Commit 与 Push 为什么是两步？

Commit 可以在离线状态下进行，是本地历史；Push 需要联网，把一个或多个本地提交发送到 GitHub。

示意：本地做了两次提交后统一上传

修改 A → Commit A

修改 B → Commit B

点击 Push origin → 两个提交一起上传到 GitHub

Fetch 与 Pull 的区别

Fetch 只询问远程“有没有新版本”；**Pull** 才会把新版本合并进当前本地分支。新手可以先 Fetch，看到 Pull origin 后再决定是否拉取。

3. 第一次准备项目

步骤 A: Fork 原作者仓库

1. 登录 GitHub，打开原作者仓库。
2. 点击右上角 **Fork**。
3. 确认 Owner 是你的账号。
4. 点击 Create fork。

完成后，浏览器地址应显示你的用户名，而不是原作者用户名。

步骤 B: 用 GitHub Desktop Clone

1. 打开 GitHub Desktop。
2. 选择 **File → Clone repository**。
3. 在 GitHub.com 标签找到你的 Fork。
4. 选择本地保存目录并点击 Clone。

示例本地路径（这是路径，不是命令）

D:\MATLABProjects\Digital-Signal-Processing-Methods-for-Ultrasonic-Echoes

步骤 C: 确认没有选错

- Current repository 显示正确项目。
- Current branch 通常显示 main。
- Repository → Repository settings 中远程地址属于你的账号。
- Repository → Show in Explorer 能打开正确目录。

不要重复 Clone 到同一个目录。已经 Clone 的项目，之后只需 Pull 更新。

4. GitHub Desktop 主界面

顶部三个区域

- **Current repository**: 当前项目，可切换仓库。
- **Current branch**: 当前分支，可创建或切换分支。
- **Fetch/Pull/Push origin**: 与 GitHub 同步。

Changes 页面

显示尚未提交的本地变化。文件前的勾选框决定该文件是否包含在下一次 Commit 中。

- 绿色行、加号 +: 新增内容。
- 红色行、减号 -: 删除内容。
- 黄色圆点: 文件被修改。

History 页面

显示已经 Commit 的历史。选择某次提交，可以查看它修改了哪些文件和代码。

左下角提交区域

Summary 写简短标题；**Description** 可写原因、测试结果或背景。点击 Commit 只创建本地记录，随后通常还需要 Push。

好的提交标题

Add README for zheng experiments
Fix syntax error in main_practice
Document Octave setup

不推荐的提交标题

改一下
update
123

5. 推荐的日常操作流程

开始工作前

1. 打开 GitHub Desktop，确认仓库。
2. 确认当前分支。
3. 点击 Fetch origin。
4. 如果出现 Pull origin，先点击 Pull。
5. 再使用 MATLAB、VS Code 或文件管理器修改文件。

修改完成后

1. 回到 Changes 页面。
2. 逐个检查修改，取消勾选无关文件。
3. 运行程序或测试。
4. 填写清晰的提交标题。
5. 点击 Commit。
6. 点击 Push origin。
7. 打开 GitHub 网站确认提交出现。

Pull → 修改 → 测试 → 检查 Changes → Commit → Push

一次提交应该多大？

一次提交只完成一件明确的事情。例如“修复语法错误”和“重写 TV 算法”应分成两个提交。这样更容易审查、撤销和定位问题。

提交前四问：文件选对了吗？有没有密码？程序测试了吗？提交说明能让别人理解吗？

6. 分支：安全的实验空间

分支是一条独立的修改路线。main 通常保留稳定版本，新功能和修复放在单独分支。

在 GitHub Desktop 创建分支

1. 点击 Current branch。
2. 选择 New branch。
3. 输入简短名称。
4. 确认基于 main 创建。
5. 点击 Create branch。

推荐的分支名称

fix-main-practice-syntax
fix-tv-denoising
add-snr-experiment
docs-octave-setup

分支工作流程

main → 创建功能分支 → 修改 → Commit → Push → Pull Request

创建后，顶部 Current branch 应显示新分支。Push 时会上传该分支，不会直接改变 main。

切换分支前最好先 Commit 当前修改。如果存在未提交修改，GitHub Desktop 可能询问把修改留在哪个分支；不确定时先取消，备份文件后再处理。

7. Pull Request：请求合并修改

Pull Request (PR) 不是“下载”，而是向仓库维护者提出合并请求。

向原作者贡献的完整流程

1. 从自己的 main 创建功能分支。
2. 完成修改并测试。
3. Commit 并 Push 功能分支。
4. 打开 GitHub 网站上的 Fork。
5. 点击 Contribute → Open pull request。
6. 确认 base repository 是原作者仓库，head repository 是你的 Fork。
7. 填写标题、修改原因、测试方法和结果。
8. 提交 PR，等待审查。

PR 描述示例

```
## 修改内容
- 修复 main_practice.m 第 32 行括号错误
```

```
## 原因
- 原代码无法被 MATLAB/Octave 解析
```

```
## 验证
- 使用 Octave 10.3 运行语法检查通过
```

原作者可以评论、要求修改、关闭或合并 PR。合并前，你的修改不会进入原作者的主仓库。

公开 Fork 中的内容任何人都可能访问；“不会自动进入原仓库”不等于“原作者看不到你的公开仓库”。

8. 同步原作者后续更新

你的 Fork 创建后不会永久自动跟随原作者。如果原项目继续更新，你需要同步。

简单方法：GitHub 网站 Sync fork

1. 打开你在 GitHub 上的 Fork。
2. 点击 Sync fork。
3. 查看差异，点击 Update branch。
4. 回到 GitHub Desktop。
5. Fetch origin，然后 Pull origin。

同步前的注意事项

- 先 Commit 或备份本地重要修改。
- 确认当前分支正确。

- 如果自己的 main 改动较多，可能产生冲突。
- 大型改动应在功能分支上完成，让 main 尽量接近上游。

若 GitHub 页面没有 Sync fork，可能是当前 Fork 已经最新，或仓库关系不是标准 Fork。

9. 冲突：两边改了同一处

当本地和远程修改了同一文件的同一位置，Git 无法自动判断保留哪一份，就会产生冲突。

看到冲突时先做什么

1. 不要连续重复 Pull 或 Push。
2. 不要删除整个仓库。
3. 先复制重要文件到仓库外作为备份。
4. 查看 GitHub Desktop 列出的冲突文件。
5. 打开文件，选择正确内容。

冲突标记示例

```
<<<<<< HEAD
你电脑上的内容
=====
GitHub 上的内容
>>>>>> origin/main
```

编辑时删除三条标记线，只留下最终需要的内容。保存后回到 GitHub Desktop，确认冲突已解决，再 Commit 和 Push。

不要盲目选择 “Use mine” 或 “Use theirs”。 先理解两边内容；有时需要把两边合理合并。

10. 撤销与恢复

场景	建议操作	风险
尚未 Commit，想丢弃某文件修改	右键文件 → Discard changes	可能永久丢失，先备份
刚 Commit，尚未 Push	Repository → Undo most recent commit	修改通常回到 Changes
已经 Push，想撤销某次提交	History → Revert changes in commit	创建反向提交，历史仍保留
不确定如何处理	先复制整个项目或关键文件到仓库外	最安全

Discard changes 是危险操作。不要把“撤销文件修改”和“撤销提交”混为一谈。

为什么推荐 Revert?

Revert 不会改写已经公开的历史，而是增加一个新提交来抵消旧提交。多人协作时通常比强制覆盖历史更安全。

11. 文件、空目录与 .gitignore

为什么空文件夹不会上传?

Git 跟踪文件内容，不单独跟踪目录。空的 zheng 文件夹不会出现在 GitHub。常见做法是在其中放置一个空的 .gitkeep 文件。

目录结构示例

```
zheng/
└── .gitkeep
```

.gitignore 的作用

.gitignore 用来排除不应提交的生成文件、缓存和本地配置。

MATLAB 项目的常见规则

```
*.asv  
*.m~  
*.fig  
slprj/  
*.mex*
```

已经被 Git 跟踪的文件，不会因为后来加入 .gitignore 就自动消失。

绝对不要上传

- 密码、Token、API Key、私钥。
- 包含个人隐私的数据。
- 没有授权的大型数据集或论文附件。
- 软件生成的缓存和临时文件。

12. 完整跟练：添加实验说明

目标：在 zheng 文件夹中添加 README，通过分支提交并上传。

第 1 步：同步

在 GitHub Desktop 中选择正确仓库和 main，点击 Fetch origin；如出现 Pull origin，再点击 Pull。

第 2 步：创建分支

Current branch → New branch，名称填写：

docs-zheng-readme

第 3 步：创建文件

在文件资源管理器进入 zheng，创建 README.md，内容如下：

```
# Zheng Experiments
```

```
This folder contains my ultrasonic signal-processing experiments.
```

```
## Goals
```

- Compare denoising methods
- Record experiment parameters
- Save reproducible results

第 4 步：检查并提交

回到 Changes，确认只有 README.md；填写提交标题：

Add README for zheng experiments

点击 Commit，然后点击 Publish branch 或 Push origin。

第 5 步：确认结果

打开 GitHub 网站，切换到 docs-zheng-readme 分支确认文件。需要合并时创建 Pull Request。

13. 常见问题

修改后为什么 GitHub 网站看不到？

通常是只修改了文件，尚未 Commit；或者已 Commit，但尚未 Push。

GitHub 有更新，电脑为什么没有？

需要 Fetch，再 Pull。浏览器中的文件不会自动出现在 Windows 文件夹。

为什么我看不到空文件夹？

Git 不跟踪空目录。加入 README.md 或 .gitkeep 后再 Commit、Push。

Push 被拒绝怎么办？

常见原因是远程有新提交。先 Fetch/Pull，解决可能的冲突，再 Push。也可能是你没有该仓库写权限。

原作者能看到我的修改吗？

如果 Fork 是公开的，原作者主动访问时可以看到；但修改不会自动进入他的仓库。创建 PR 后，原作者会更明确地收到通知。

可以直接在 main 修改吗？

小型个人练习可以，但正式协作推荐建立功能分支。

Commit 以后还能改吗？

可以。你可以继续修改并创建新 Commit；若要撤销已公开提交，优先使用 Revert。

14. 操作速查表

我想做什么	应该使用
复制别人的项目到自己账号	Fork
首次下载自己的仓库	Clone
检查 GitHub 是否有更新	Fetch origin
下载并合并 GitHub 更新	Pull origin
记录本地修改	Commit
上传本地提交	Push origin
隔离一个新功能或实验	New branch
请求原作者合并修改	Pull Request
安全撤销已公开提交	Revert
打开项目目录	Repository → Show in Explorer

最终记忆口诀

开始先拉取，修改后测试；检查再提交，提交后推送；大改用分支，贡献发 PR。

每次 Push 前检查

- 仓库和分支正确。
- 没有密码和隐私信息。
- 没有无关的大文件。
- 程序或测试已运行。
- 提交说明清晰。

本教程以 GitHub Desktop 的常见版本为例。界面文字可能随版本略有变化，但 Git 的核心概念与流程不变。